

CSCI 361 Lecture 19:

More NP Complete Problems

Shikha Singh, Sam McCauley

Announcements & Logistics

- Hand in **Exercise #15**, no Exercise for Thursday's class
- **HW 9** is due this Thursday at 10 pm
 - Partner assignment (may work in pairs, not necessary)
 - Submit a single PDF to Gradescope
- Shikha has virtual office hours slots you can reserve:
 - Tues/Wed/Thur 2-3 pm EST
 - Use link: <https://calendar.app.google/oLLLwVw4uMjrwFxy7>
 - Can also ask Sam questions in the last 10 mins of lecture

Last Time

- Discussed definition of NP hard and NP Complete problems
- Proved that SAT and 3SAT are NP Complete
 - Any language $L \in \mathbf{NP}$ can be reduced in poly-time to **SAT**
 - Encoded a NTM's computation table as a SAT instance
- Practice with reductions:
 - $3\text{SAT} \leq_p \text{CLIQUE}$
 - $\text{CLIQUE} \leq_p \text{INDSET}$

Class Exercise Solutions

- **Exercise 1.** $\text{CLIQUE} \leq_p \text{IND-SET}$:
 - Given an instance $\langle G = (V, E), k \rangle$ of Clique, create a new graph $G' = (V, \bar{E})$ where $(u, v) \in \bar{E}$ iff $(u, v) \notin E$ for each pair $u, v \in V$
 - Then G has a k -clique iff G' has an independent set of size k
- **Exercise 2.** $\text{HamCycle} \leq_p \text{TonianCycle}$, where **tonian cycle** in a graph G is a cycle that visits at least half of the vertices of G
 - Given a instance $\langle G \rangle$ of HamCycle with n nodes, add n dummy (isolated) nodes to G'
 - Then G has a Hamiltonian cycle iff G' has a tonian cycle.

Any questions on these reductions?

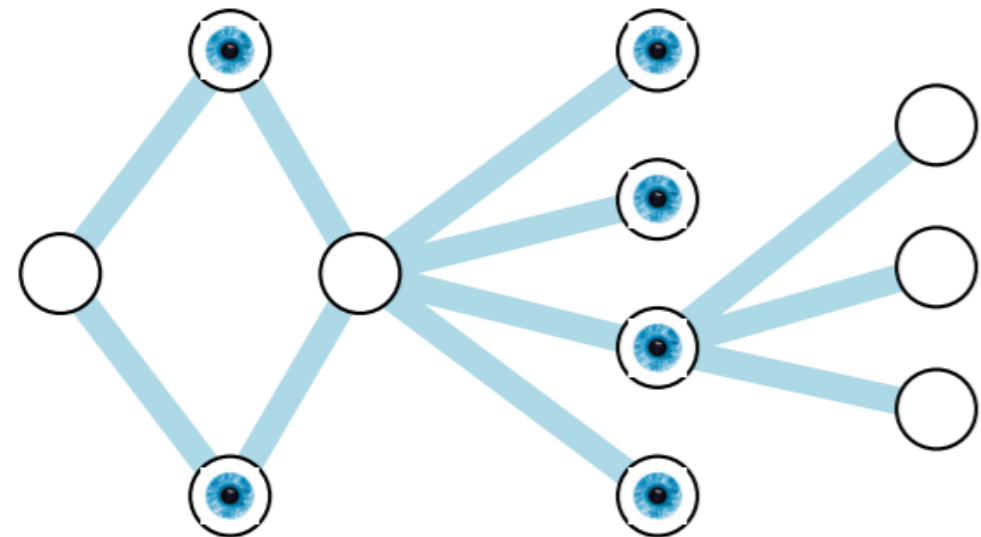
Today

- More practice with NP hardness reductions
- Prove $3\text{SAT} \leq_p \text{Vertex-Cover}$ and $3\text{SAT} \leq_p \text{HAMPATH}$
- If there is time, will show $3\text{SAT} \leq_p 3\text{COLOR}$
- Leave time for group practice problems/questions on HW 9

Vertex Cover is NP Complete

- **Vertex cover** of a graph $G = (V, E)$ is a subset $C \subseteq V$ such that each edge $e \in E$ has at least one end point in C

VERTEX-COVER = $\{ \langle G, k \rangle \mid G \text{ has a vertex cover of size } k \}$



A vertex cover of size 6

Vertex Cover is NP Complete

- To show vertex cover is NP complete, need to show:
 - Vertex Cover is in NP
 - Show that a **known** NP hard problem reduces to it via a polynomial-time reduction
- Can show **INDSET** $\leq_p$ VertexCover, since we proved **INDSET** is NP hard last class
- **Different strategy:** Let's reduce directly from 3SAT using **variable and clause gadgets** to practice such reductions

Map the Problems

3SAT

Variables and clauses

What is an input?

Vertex-Cover

Vertices and edges

What is a possible solution?

A selection of vertices set to TRUE

A selection of vertices in the cover

What is the requirement?

Each clause has at least 1 True literal

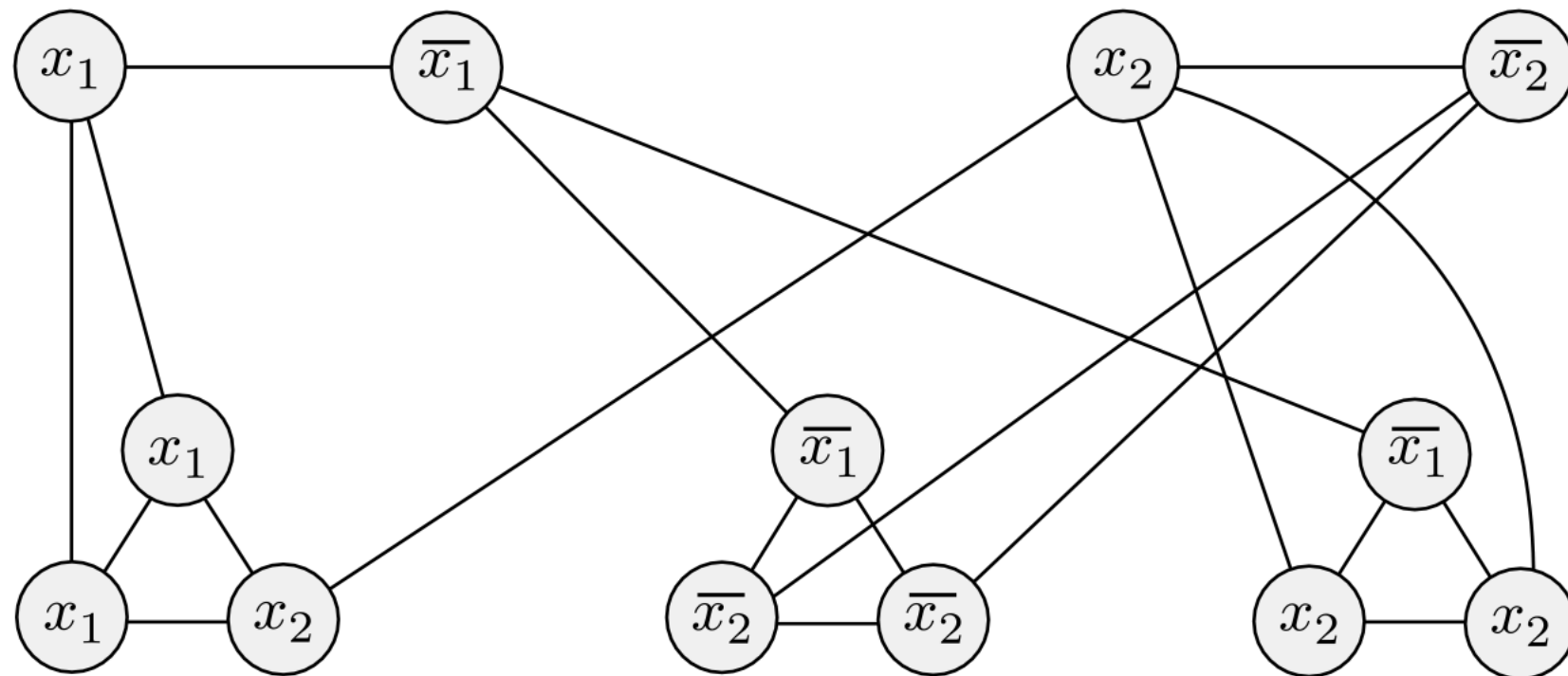
Each edge must have at least one
end point in cover

3SAT to Vertex Cover

- Given a 3SAT instance ϕ , create a graph $G = (V, E)$ as follows:
 - For each variable x in ϕ , create a node x and $\bar{x}$ and connect them with an edge (**variable gadget**)
 - For each clause c in ϕ , create a "triangle" of 3 nodes, each labeled with the corresponding literal and connected to each other and to the nodes in the variable gadget with the identical label (**clause gadget**)
- Instance of VC is $\langle G, k \rangle$ so we need to set k
 - Easier to "backward engineer" value of k after thinking through the reduction

3SAT to Vertex Cover

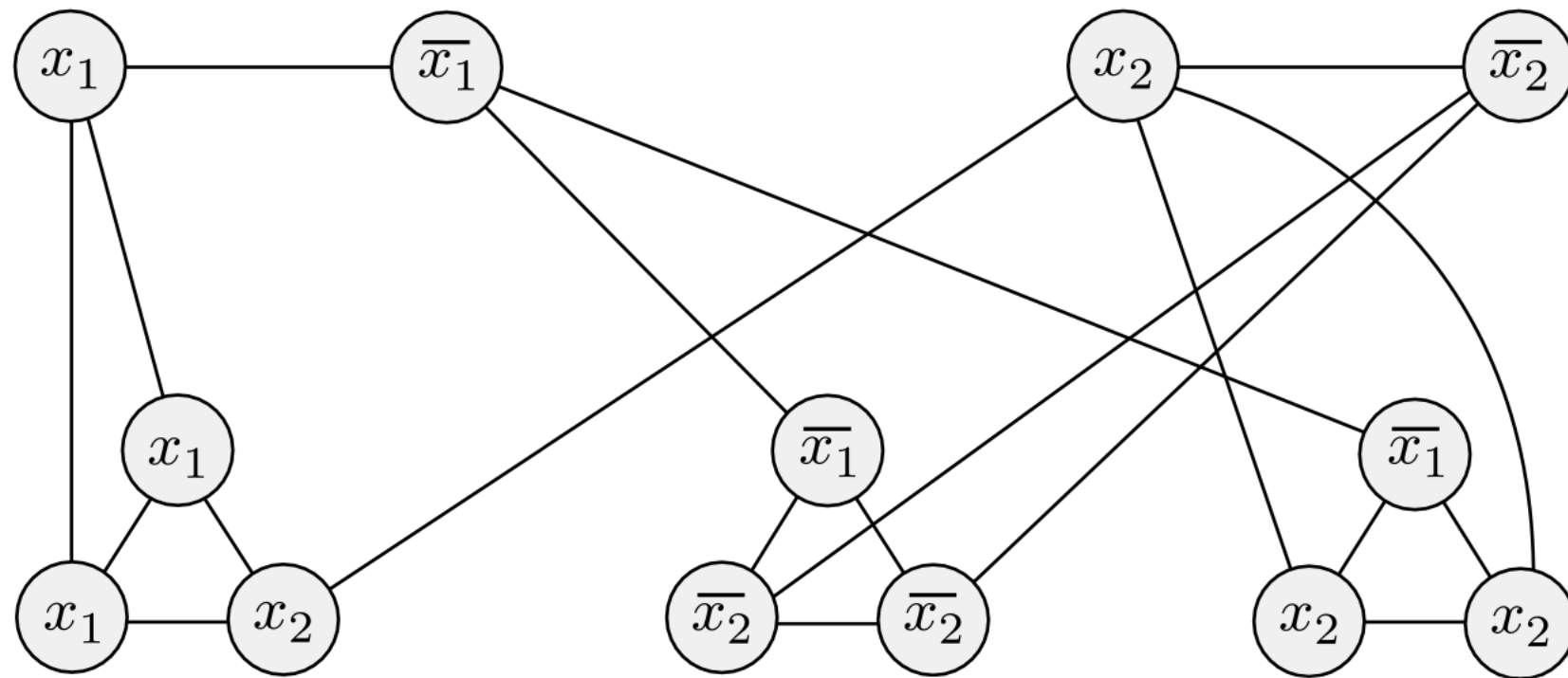
$$\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$$



- If ϕ has m variables and ℓ clauses, what is the # of vertices & edges in G ?
- $2m + 3\ell$ nodes, $m + 6\ell$ edges

3SAT to Vertex Cover

$$\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$$



- To cover edges in each triangle gadget, need to include at least 2 vertices in the cover
- Moreover, to cover the vertex-gadget edge, we need one node from each such gadget in the cover, so let's try $k = m + 2\ell$

Correctness of Reduction

- Set $k = m + 2\ell$
- ($\Rightarrow$) If ϕ is satisfiable, then G has a k -vertex cover
 - From each variable gadget, pick the node corresponding to the true literal in the cover C
 - Each clause gadget has at least one **T** literal, choose the other two vertices in the cover C from each gadget
 - Clearly, $|C| = k$. Why is C a valid vertex cover?
 - Why are all the edges between variable and clause gadgets covered?

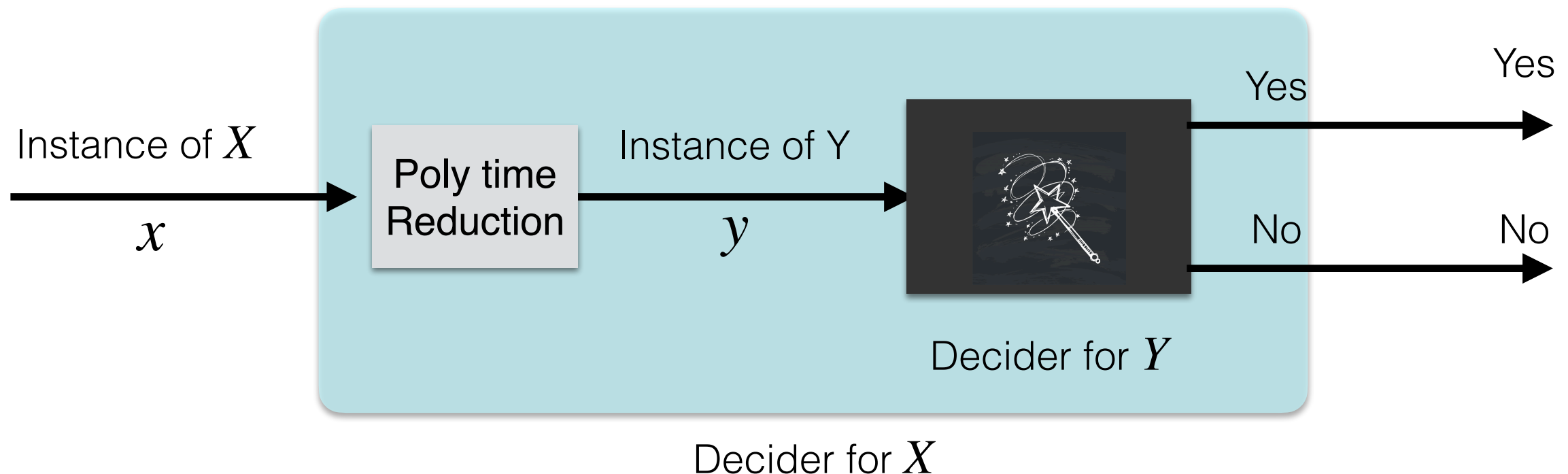
Correctness of Reduction

- Set $k = m + 2\ell$
- ($\Leftarrow$) If G has a vertex cover C of size k , then ϕ is satisfiable.
 - To account for $m + 2\ell$ nodes in C , what must it include?
 - Must contain one vertex from each variable gadget and two per clause gadget.
 - For each variable gadget node in C , set the corresponding literal to True
 - Why is this a satisfying assignment of ϕ ?

Reductions Quiz

Say $X \leq_p Y$. Which of the following can we infer?

- If X can be solved in polynomial time, then so can Y .
- X can be solved in poly time iff Y can be solved in poly time.
- If X cannot be solved in polynomial time, then neither can Y .
- If Y cannot be solved in polynomial time, then neither can X .



Digging Deeper

- If $X \leq_p Y$ and X can be solved in polynomial time, this does not imply that Y can be solved in polynomial time.
- Counter-example is in **HW 9**, where you are asked to show:
 - Graph 2-Color reduces to Graph 3-color
 - Graph 2-Color can be solved in polynomial time
- Graph 3-color (we'll show) is NP hard and unlikely to have a polynomial-time solution

Intuitively, if problem X reduces to problem Y ,
then solving X is no harder than solving Y

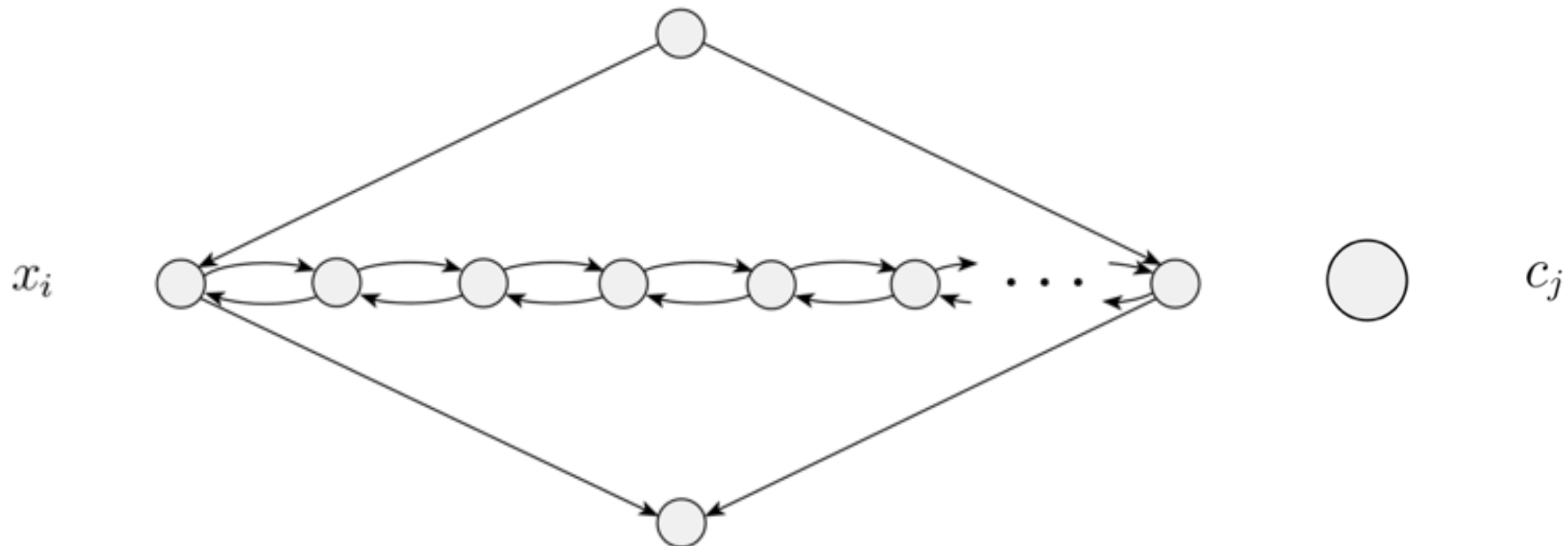
$$3\text{SAT} \leq_p \text{HAMPATH}$$

$$3\text{SAT} \leq_p \text{HAMPATH}$$

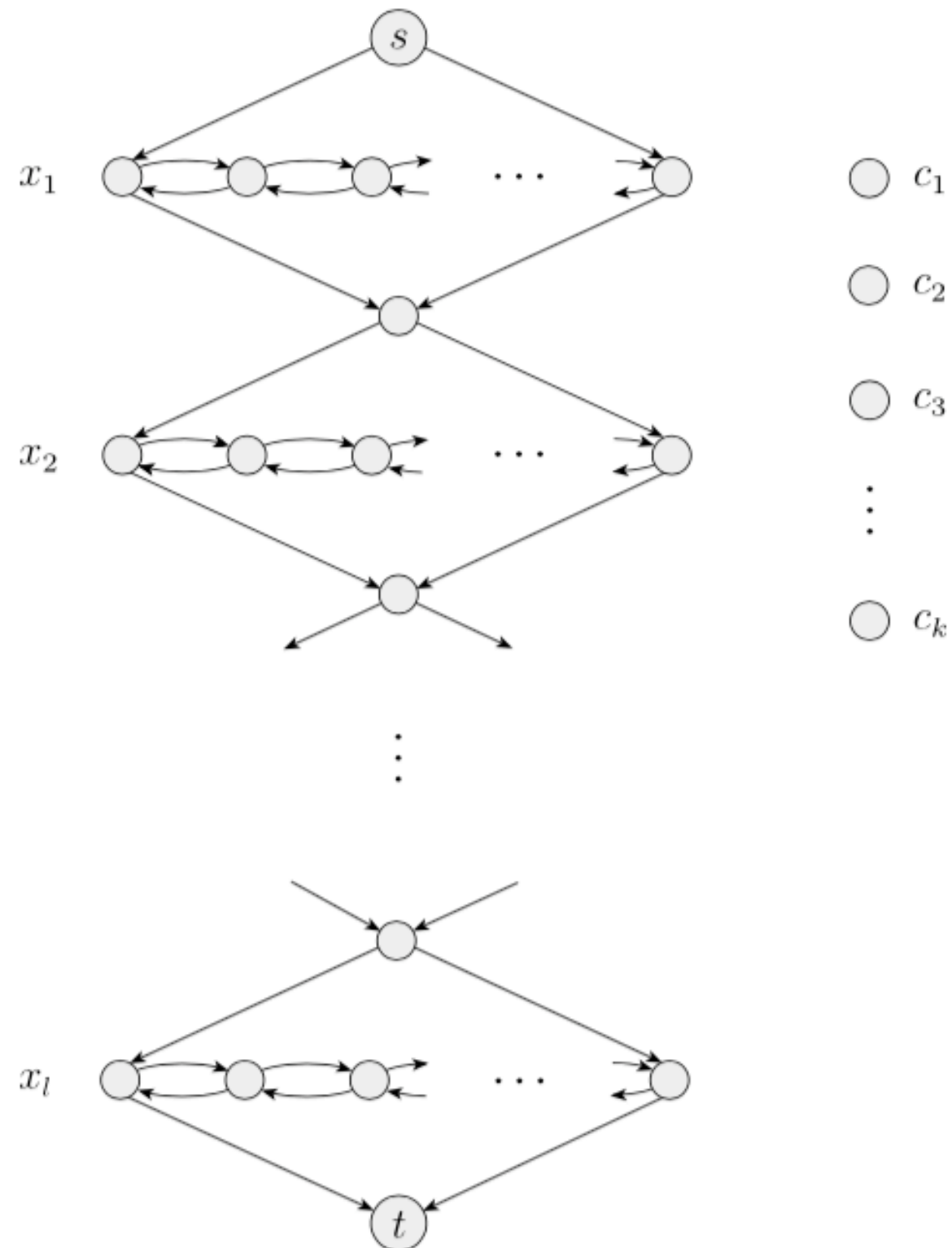
- Given 3SAT instance Φ , transform it to directed graph G with nodes s, t such that Φ is satisfiable iff G has a hamiltonian path from s to t (a path that visits each node exactly once)
- Essential ingredients of a input assignments of Φ
 - Each variable can be set to true or false (need to encode these settings in the graph in our variable gadget)
 - For a clause to be satisfied at least one literal is set to true
 - Need to hook up clause and variable gadgets to ensure iff correctness

$3\text{SAT} \leq_p \text{HAMPATH}$

- Let Φ contain k clauses and ℓ variables
- Let $x_1, \dots, x_\ell$ denote the ℓ variable in Φ
- **Variable gadget:** for each variable x_i create a diamond shape structure with a horizontal row of nodes; one for each clause
- **Clause gadget:** for each clause c_j we create a single node



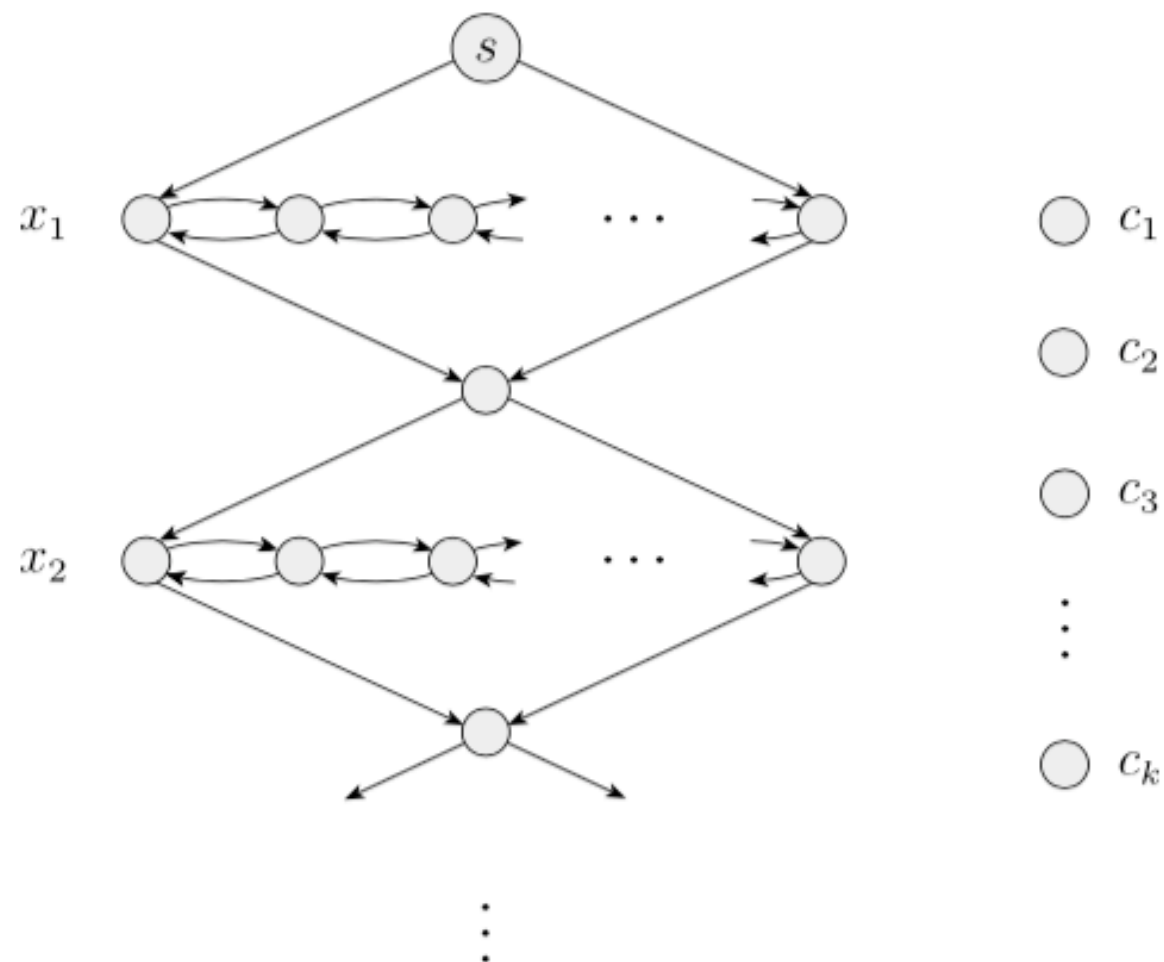
$$3\text{SAT} \leq_p \text{HAMPATH}$$



$3SAT \leq_p HAMPATH$

Proof so far:

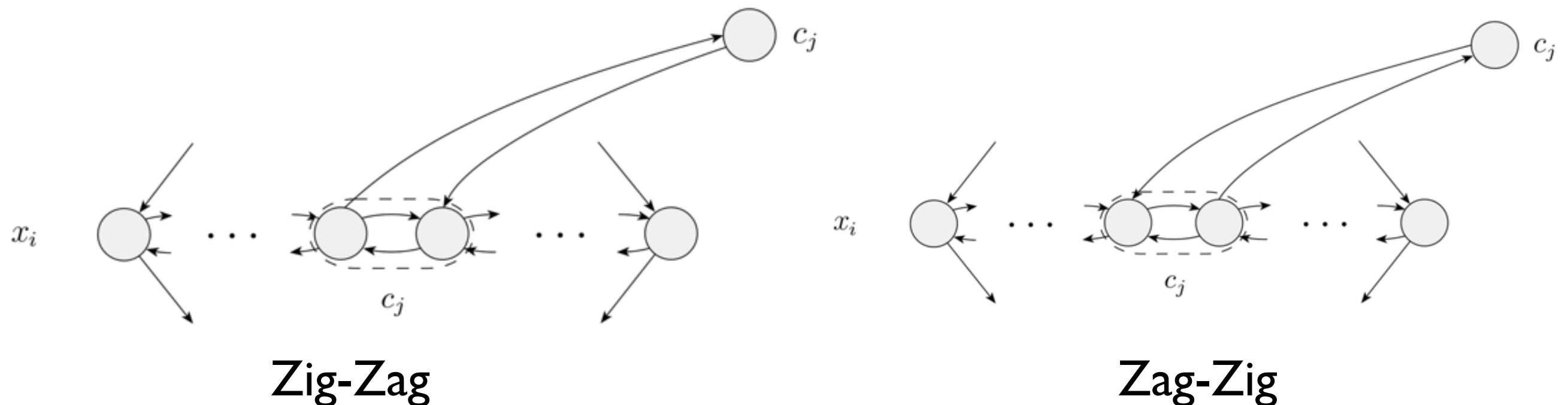
- Must traverse middle part of each diamond either **right** or **left**
- “Traversing **right**” = variable is **true**
- “Traversing **left**” = variable is **false**
- Now: each variable is true or false. How can we ensure that each clause has a true literal?



$3\text{SAT} \leq_p \text{HAMPATH}$

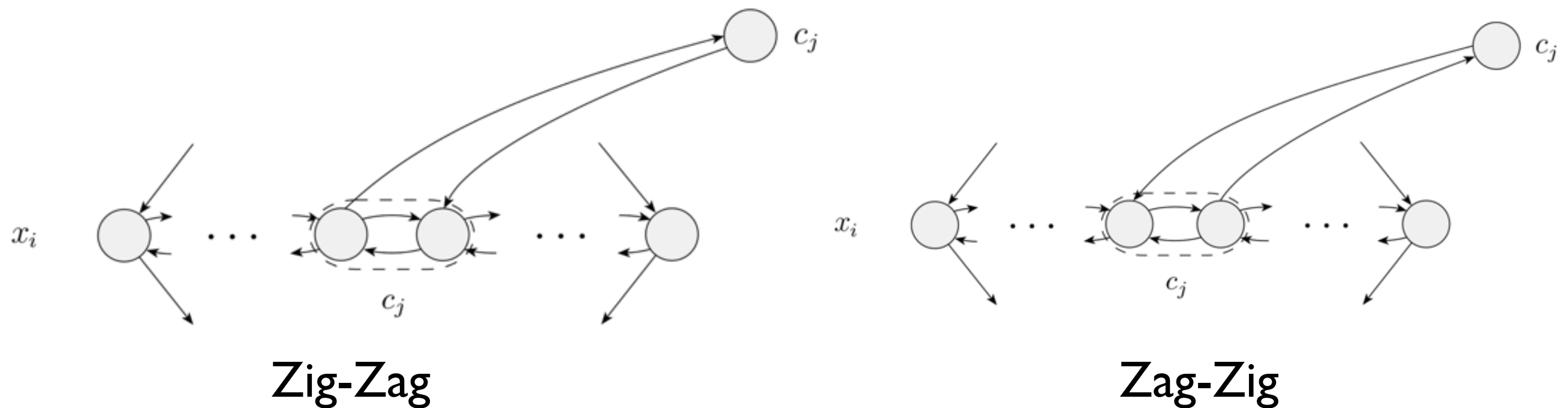
Connecting the variable and clause gadgets.

- If x_i appears in c_j , connect j th pair in the i th diamond to the j th clause: connect in a zig-zag fashion
- If $\bar{x}_i$ appears in c_j , connect it in a zag-zig fashion (left)
- If neither, no connection from the diamond to c_j



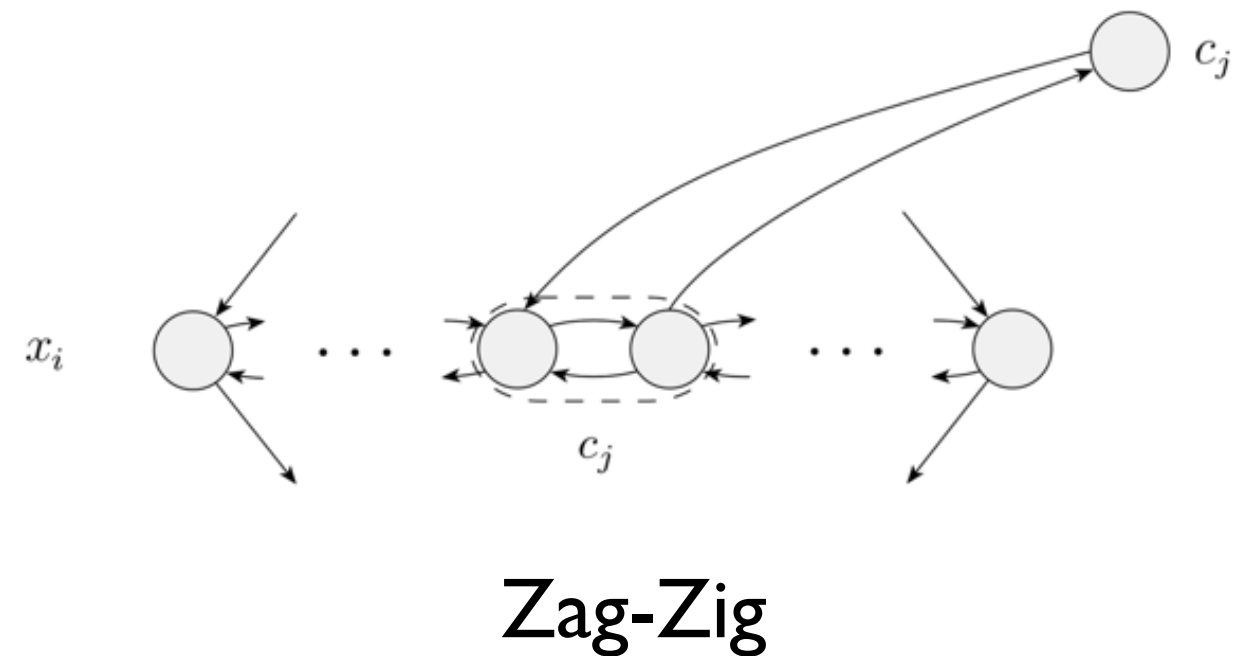
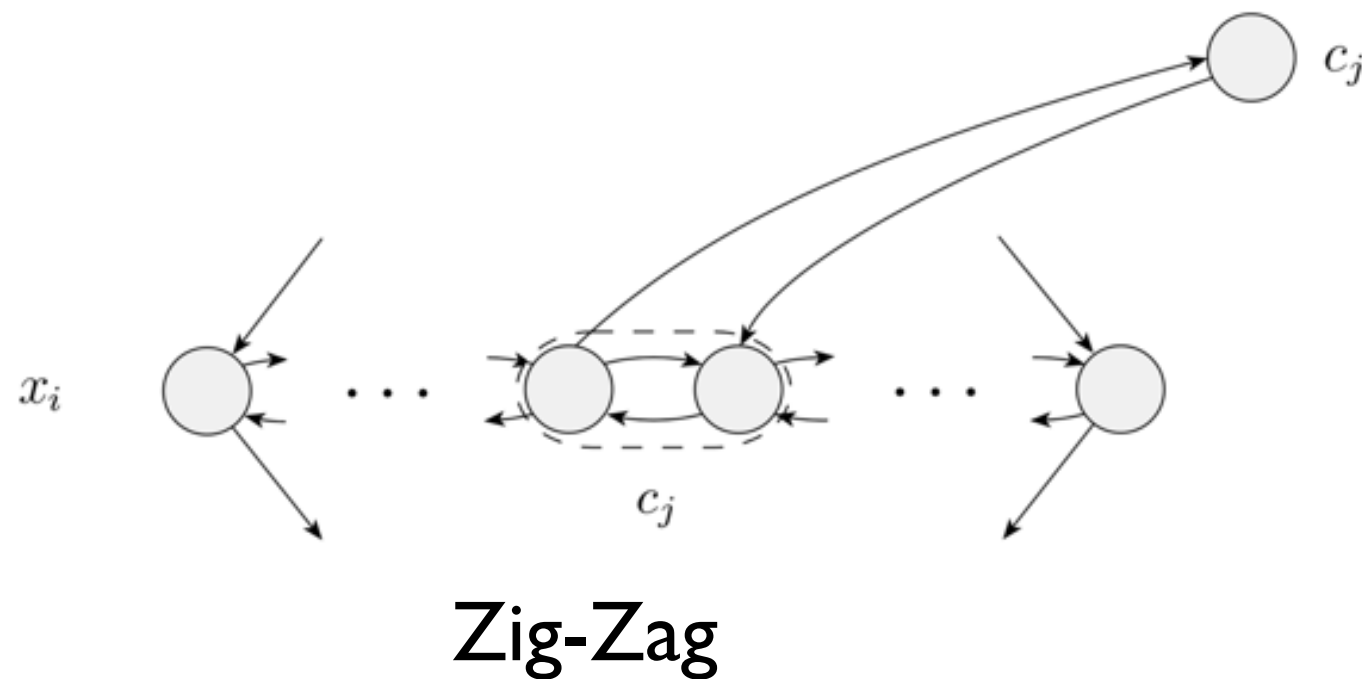
$3SAT \leq_p HAMPATH$

- If x_i is true, the diamond is traversed to the right, and the Hamiltonian cycle can visit c_j in a zig-zag way
- Else if $\bar{x}_i$ is true, the diamond is traversed to the left, and the Hamiltonian cycle can visit c_j in a zag-zig way



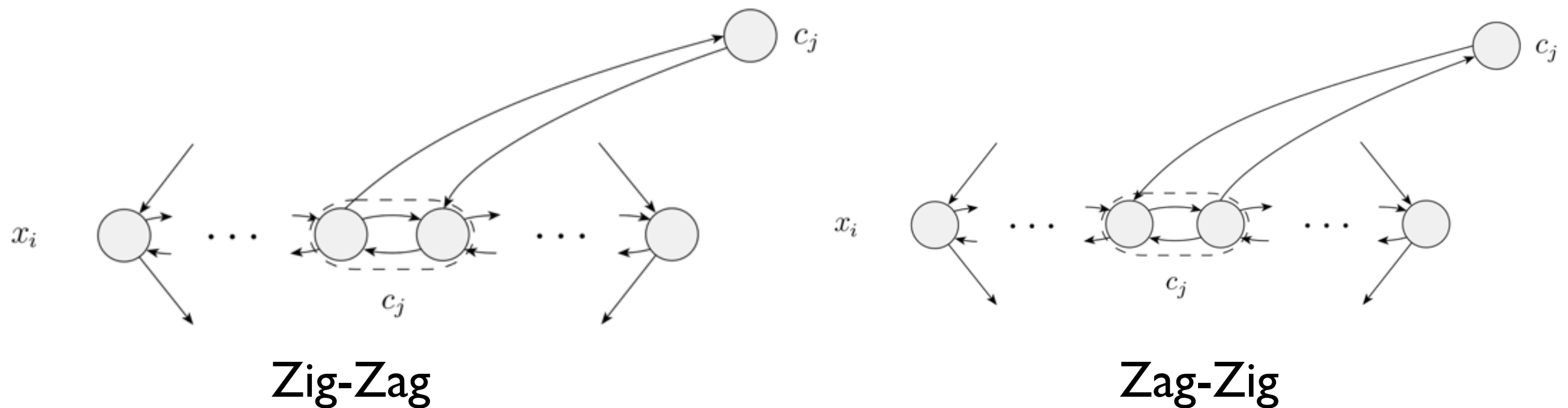
$$3\text{SAT} \leq_p \text{HAMPATH}$$

- The node for a clause variable can only be visited if the corresponding literal is true!
- So all clauses must be satisfied



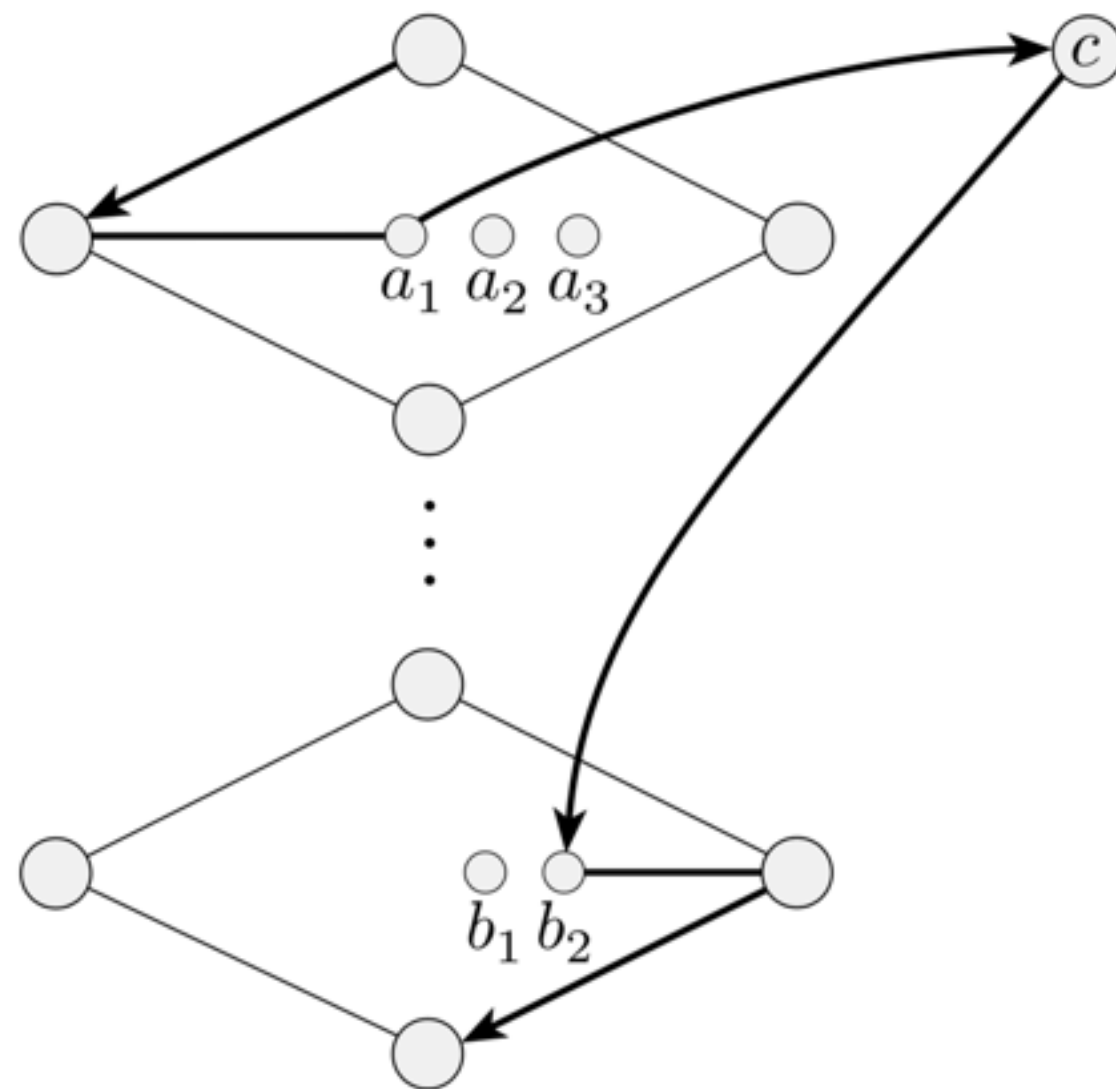
$3\text{SAT} \leq_p \text{HAMPATH}$

- Wait a minute: what makes us go back to the same diamond after visiting a clause?
- Our proof assumed that we visited the diamonds one at a time. Can some crazy path allow us to “jump around” using clauses?



$$3\text{SAT} \leq_p \text{HAMPATH}$$

- Situation that cannot occur in a Hamiltonian cycle of G : clause entered from one diamond but exited to a different



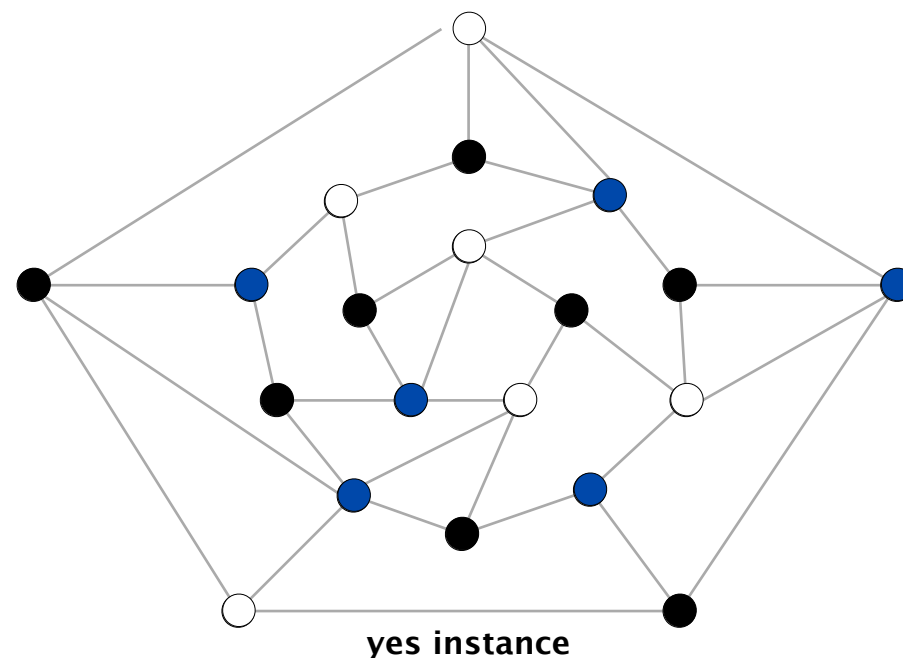
Such a cycle would never visit node a_2 — the only nodes with edges into a_2 are c and a_1

Graph-3-Color is NP Complete:

$$3\text{-SAT} \leq_p \text{Graph 3-Color}$$

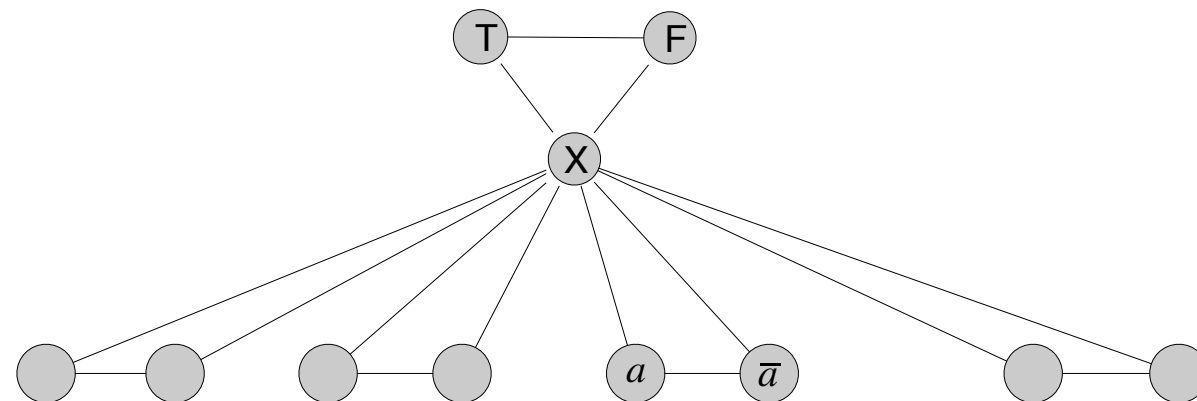
Graph 3-Color Problem

- **3-COLOR.** Given an undirected graph $G = (V, E)$, is it possible to color the vertices with 3 colors s.t. no adjacent nodes have the same color.
- We have shown in the past that 3-COLOR $\in \text{NP}$.



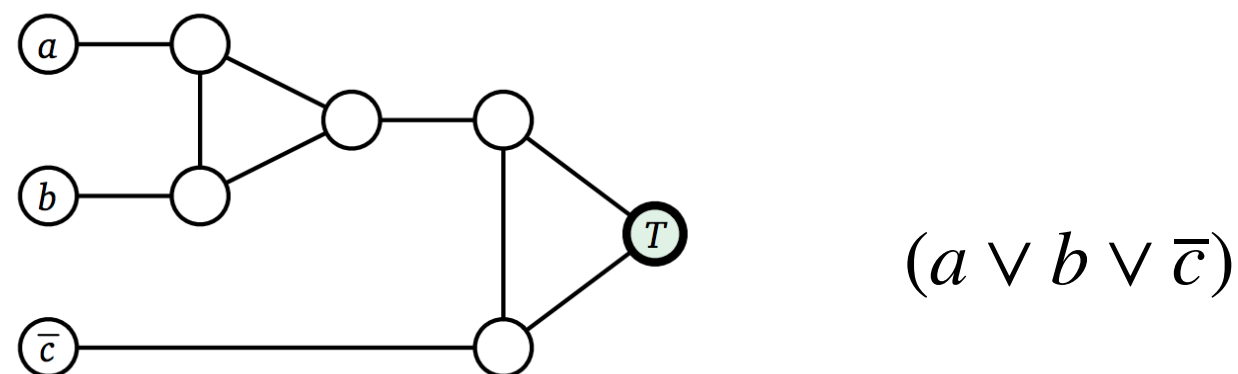
$$3\text{-SAT} \leq_p 3\text{-COLOR}$$

- **Proof.** Given a 3-SAT instance Φ , define G as follows
 - **Truth gadget:** a triangle with three nodes T, F , and X (for true, false and other) — they must get different colors (say true, false, other)
 - **Variable gadget:** a triangle made up of variable a , its negation $\bar{a}$ and the X node of the truth gadget — enforces $a, \bar{a}$ are colored true/false



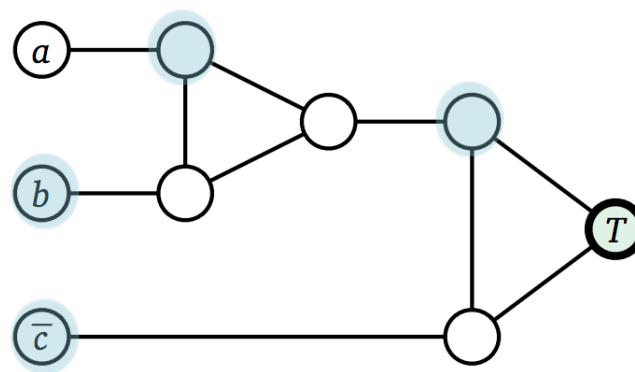
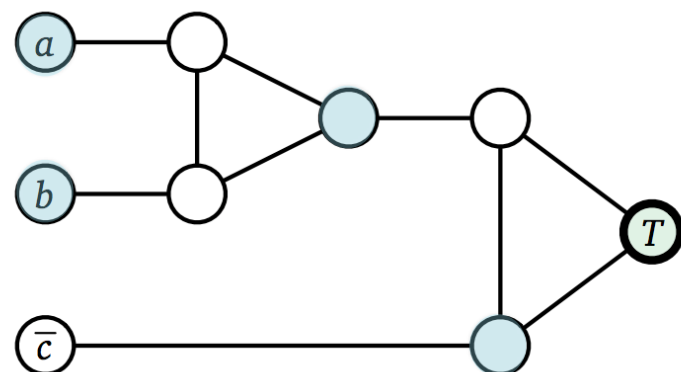
$$3\text{-SAT} \leq_p 3\text{-COLOR}$$

- Given a **3-SAT** instance Φ , define G as follows
 - Truth gadget**: a triangle with three nodes T, F , and X (for true, false and other) — they must get different colors (say true, false, other)
 - Variable gadget**: triangle made up of variable a , its negation $\bar{a}$ and the X node of the truth gadget — enforces $a, \bar{a}$ are colored true/false
 - Clause gadget**: joins three literal nodes (from the variable gadget) to node T in the truth gadget using a subgraph as shown below

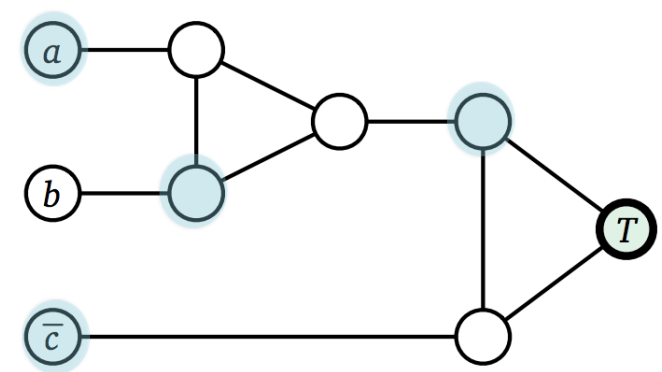


$3\text{-SAT} \leq_p 3\text{-COLOR}$

- Observation. A clause gadget has a valid 3-coloring, if and only if, at least one literal is colored True
 - If a, b (or $b, \bar{c}$) or $(a, \bar{c})$ get the same color (say, FALSE) then the right-end-point of the triangle must be colored the same (shown in blue)
 - The remaining literal must be colored True!



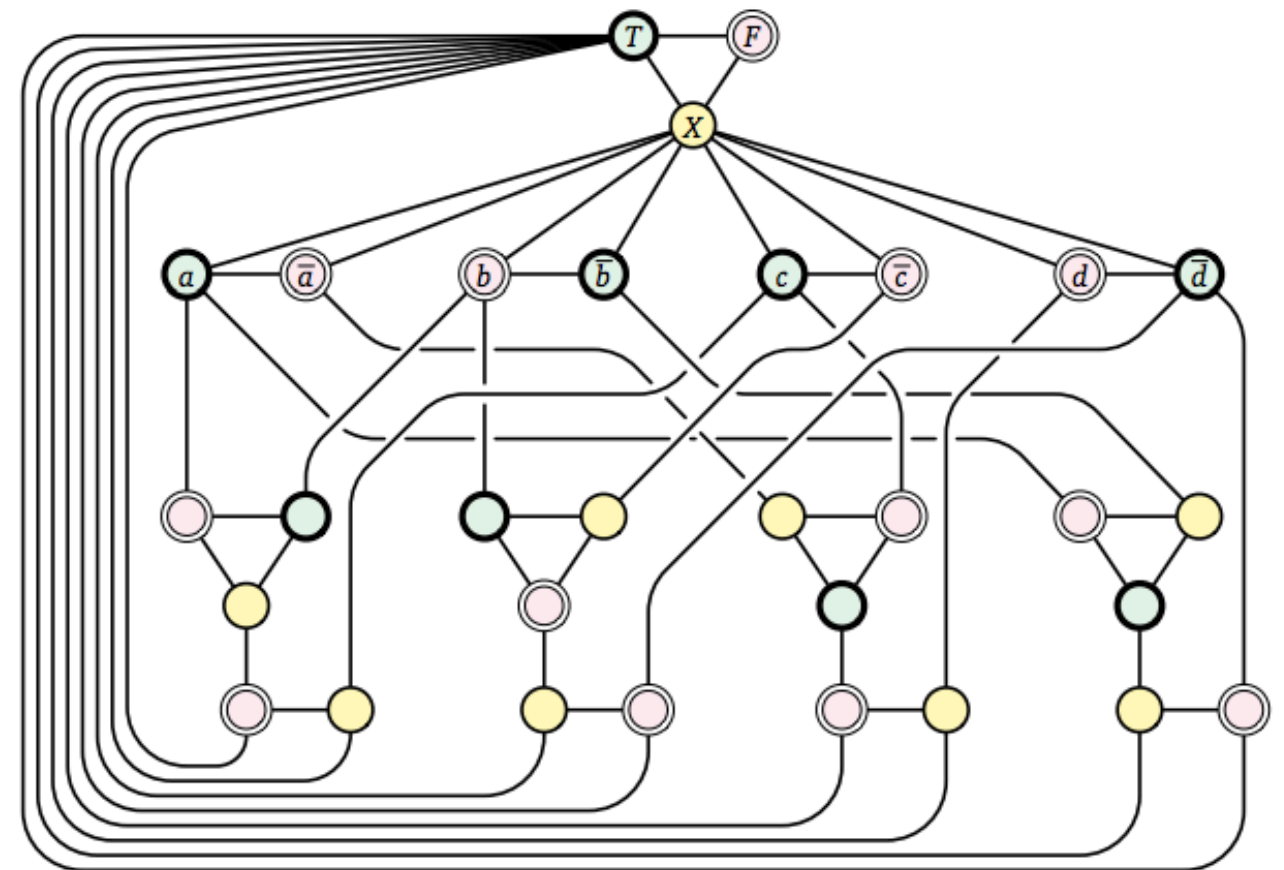
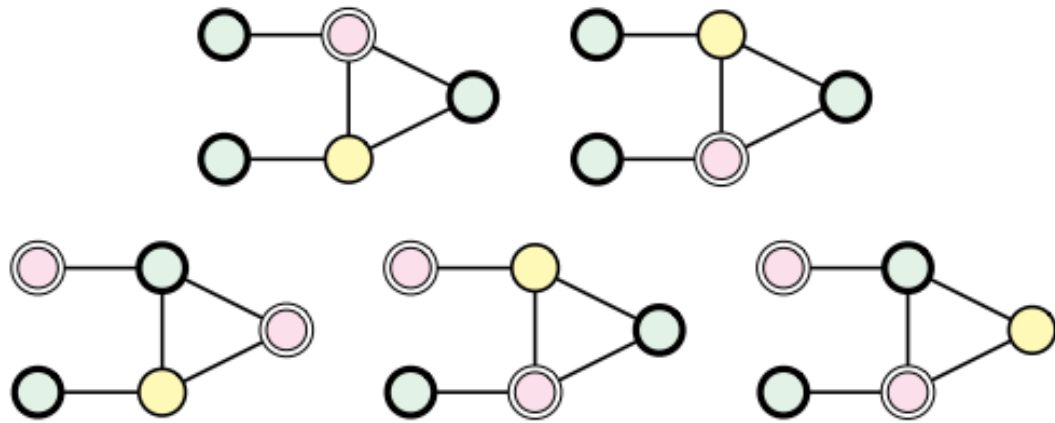
$(a \vee b \vee \bar{c})$



$3\text{-SAT} \leq_p 3\text{-COLOR}$

- Example. All valid 3-colorings of the “half-gadget” of the clause on the left
- Overall G for example instance on the right

$$(a \vee b \vee c) \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b} \vee \bar{d})$$



$$3\text{-SAT} \leq_p 3\text{-COLOR}$$

Correctness of reduction

- ($\Rightarrow$) If Φ is satisfiable, color the variable gadget based on the satisfying assignment
- Why can we extend this coloring to a valid 3-coloring of the clause gadgets?
 - Because at least one literal in each clause must be True
- ($\Leftarrow$) If G is 3-colorable, then the variable nodes must be colored T or F (because of the variable gadget), we can assign truth values based on the colors
- Why is this a satisfying assignment?
 - At least one of the literals in each clause must be colored true and thus the resulting assignment must satisfy Φ

$$3\text{-SAT} \leq_p 3\text{-COLOR}$$

Running time of reduction

- If ϕ has n variables and k clauses, then our resulting graph G has at most $2n + 5k + 3$ nodes
- Thus, we can construct G in $O(n + k)$ time
- Reduction is polynomial time!

Class Group Exercises

Given a graph G and integer k , does G have a spanning-tree where each vertex has degree at most k .

- Recall that a spanning tree is a connected acyclic subgraph that includes all vertices
- **Exercise 1a.** State this as a decision problem and prove it is NP complete.
- **Exercise 1b.** What if I modified the problem to determine if a given graph G has a spanning tree of degree at most 13? Prove that this problem is also NP complete.
- **Exercise 2.** Prove this problem is NP complete: Given a graph G , is it possible to color all its nodes using 3 colors, such that at most $3/7$ edges have both endpoints with the same color.

Solutions

- **Exercise 1a.** Setting $k = 2$ makes it exactly the Hampath problem.
- **Exercise 1b.** Attach 11 dummy nodes to each vertex. Now any spanning tree has to include these nodes/edges. If the original graph has a hampath, then this new graph has a spanning tree of degree at most 13 and vice-versa.
- **Exercise 2.** Create a new graph G' that is G union with 317 extra K_4 (complete graph on 4 vertices).
 - Valid 3-coloring of G iff can color G' with at most 317 "bad" edges (one per K_4 graph)